CODE SPORT

This month's column continues the discussion on data storage systems, with a focus on how file systems remain consistent even in the face of errors occurring in the storage stack.

Last month we discussed the concept of scale-up and scale-out storage and their relative merits. One of the most important parts of a storage stack is the file system. In Linux, there are a number of popular file systems like *ext3*, *ext4*, *btrfs*, etc. File systems hide the complex details of the underlying storage stack such as the actual physical storage of data. They act as containers of user data and serve any IO requests from user applications.

File systems are complex pieces of software. Traditionally, they have been a kernel component. Different file systems offer different functionalities and performance. However, their interactions with user applications have been simplified through the use of the Virtual File System (VFS), which is an abstract layer sitting on top of concrete file system implementations like *ext3*, *ext4* and *ZFS*. The client application can program to the APIs exposed by the VFS and does not need to worry about the internals of the underlying concrete file systems. Of late, there has been considerable interest in developing user space file systems using the FUSE module available in mainstream Linux kernels (*fuse.sourceforge.net/*). User-space file systems can be created by using the kernel FUSE module, which intercepts the calls from the VFS layer and redirects them back to user-space file system code.

In this month's column, we look at the challenges in ensuring the safety and reliability of data in file systems.

Keeping data safe

Given the massive importance and explosion of data in this era of data driven businesses, file systems are entrusted with the important responsibility of keeping data safe, correct and consistent, forever, while ensuring high accessibility to the data. Data loss is unthinkable, while unavailability of data even for short durations can have disastrous consequences on businesses. While data has been growing at an exponential rate, the reliability of the file systems, which act as data containers, has not matched the demands of the 'always on/always consistent' data. Unfortunately, storage systems do fail and the manner in which they fail is complex—they can exhibit partial failures, failures which are recognised much after they occur, failures which are transient, and so on. Each of these different types of failures imposes different reliability requirements on file systems, which need to detect, handle and repair such failures.

Figure 1 shows a simple model of the storage system, wherein the bottom most layer is the magnetic media on which the data is physically stored, and the top most layer is the generic file system with which the client application interacts. Each block of the storage system can fail in different ways, leading to data unavailability and inconsistency. As the data flows through the storage stack, at each point it is vulnerable to errors. An error can occur in any of the layers and propagate to the file system. This results in inconsistent or incorrect data being written or being read by the user through file system interfaces.

At the bottom-most layer, disks can fail in complex ways. It is no longer possible to make